Meta最新模型LLaMA细节与代码详解

原创 常鸿宇 于 2023-03-06 19:36:40 发布 阅读量3.2w 收藏 335 点赞数 135 版权
分类专栏： 生成模型 自然语言处理 文章标签： 人工智能 深度学习 自然语言处理

GitCode 开源社区 文章已被社区收录 加入社区

自然语言处理 同时被 2 个专栏收录 61 订阅 47 篇文章 订阅专栏

Meta最新模型LLaMA细节与代码详解

0. 简介
1. 项目环境依赖
2. 模型细节
    2.1 RMS Pre-Norm
    2.2 SwiGLU激活函数
    2.3 RoPE旋转位置编码
3. 代码解读
    3.1 tokenizer
    3.2 model
        3.2.1 模型细节详解
        3.2.2 transformer构建
    3.3 generate
4. 推理

0. 简介

今天介绍的内容是Facebook Meta AI最新提出的语言模型LLaMA，该模型声称以更小的体积，在多数任务上超越了GPT-3的性能。

模型相关项目已经开源：
https://github.com/facebookresearch/llama

论文地址：https://scontent-tpe1-1.xx.fbcdn.net/v/t39.8562-6/333078981_693988129081760_4712707815225756708_n.pdf?_nc_cat=108&ccb=1-7&_nc_sid=ad8a9d&_nc_ohc=ov6yTHfLfNQAX-guxqd&_nc_ht=scontent-tpe1-1.xx&oh=00_AfDMyTEYewg-cHT9_4_sUaW5h0gqrqwjcNMylD9HtVFCWA&oe=6401C9E2

由于模型较大，目前的设备暂时没有办法支持进一步的实验，但是其模型代码已经开源，所以可以先通过代码了解一下模型结构上的一些细节，今天就针对github上放出的代码，了解一下模型的细节。

此外，该模型其实就是transformer做了一点细节上的改进，真正更有价值的工作应该在数据和训练方面。通过阅读代码，可以对transformer的基础构造进行复习，并且了解大模型如何在多卡上分布推理。
由于该项目源码几乎没有注释，这就肯定会给很多同学阅读时带来困扰，所以本文顺带着就把代码部分详细的介绍一下。

1. 项目环境依赖

此项目给出的环境依赖只有4个：

- torch
- fairscale
- fire
- sentencepiece

其中torch不比多讲，fairscale是用来做GPU分布的，一般是当使用DDP仍然遇到超显存的问题时使用fairscale。目前fairscale我还没有试过，在下文的源码介绍中，我会用torch中对应的基础网络替代fairscale中的结构层进行介绍。fire是一个命令行工具，用或者不用他都可以，sentencepiece是用于tokenizer的工具包，会在tokenizer部分简单介绍。

2. 模型细节

由于该模型就是用的transformer的decoder，所以在结构上它与GPT是非常类似的，只是有一些细节需要注意一下。

2.1 RMS Pre-Norm

关于Pre-Norm和Post-Norm是神经网络中老生常谈的话题，目前比较普遍的被大家接受的结论是，相同的深度条件下，Post-Norm的效果要优于Pre-Norm，因为Pre-Norm实际上相当于通过了一个更宽的网络而非更深的网络，所以在同等深度下，Pre-Norm的实际效果相当于一个更浅却更宽的网络，详细的推理过程参考：https://spaces.ac.cn/archives/9009。

然而在LLaMA中却采用了Pre-Norm，或许是因为模型够深（7B，13B，30B，65B的模型，transformer layer数量分别为32，40，60，80），而Pre-Norm的恒等分支更加明显，有利于梯度的传播（这部分暂时没有想到很合理的解释，如果有更好的理解，欢迎在评论区补充）。

RMS Norm（Root Mean Square Layer Normalization），是一般LayerNorm的一种变体，可以在梯度下降时令损失更加平滑。

与layerNorm相比，RMS Norm的主要区别在于去掉了减去均值的部分（re-centering），只保留方差部分（re-scaling），从归一化的表达式上可以直观地看出：

- 一般的LN：

$$\overline{a}_i = \frac{a_i - \mu}{\sigma} g_i$$

其中，

$$\mu = \frac{1}{n}\sum_{i=1}^{n} a_i$$

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(a_i - \mu)^2}$$

- RMS Norm：

$$\overline{a}_i = \frac{a_i}{RMS(a)} g_i$$

其中，

$$RMS(a) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} {a_i}^2}$$

可以看到，二者的区别就在于有没有减去均值。至于RMS Norm为什么有用，需要求梯度进行分析，感兴趣的同学可以阅读RMS Norm的论文。

2.2 SwiGLU激活函数

LLaMA采用SwiGLU替换了原有的ReLU。

采用SwiGLU的FNN，在论文中以如下公式进行表述：

$$FFN_{swiGLU}(x, W, V, W_2) = (Swish_1(xW) \otimes xV)W_2$$

其中，$Swish_\beta(x) = x\sigma(\beta x)$, (Ramachandran et al., 2017.)

2.3 RoPE旋转位置编码

RoPE（Rotary Position Embedding）旋转位置编码，是苏剑林老师提出的一种旋转位置编码方法，其思想是采用绝对位置编码的形式，实现相对位置编码。这一部分比较关键，如果不理解的话，后边的代码估计就看不懂了。读懂RoPE涉及一点复变函数的基础知识，不过如果你没有学过的话也没有关系。

位置编码对大模型而言尤为重要，因为既然是要训练大模型，那么长文本的表征和模型对于长文本的建模能力就显得非常重要。（*但是对于绝对位置编码，我有一个直观地感受，认为其本质上不适用于长文本的场景，因为它会直接导致模型的Embedding层被无限放大，并且由于数据分布在seq_len方向上通常是长尾的，这又会必然导致绝对位置编码的矩阵在尾部会越来越稀疏，一方面造成资源浪费，另一方面这种表示方法直观上就很不利于模型的学习，因为它与我们实际场景是有很大的矛盾的。而RoPE虽然具有相对位置编码的性质，但是从代码部分可以看出，在构造的时候，其也是受到了最大长度的限制的。关于这一点，我无法严谨得说明，只是一点个人的想法。*）。

而RoPE的巧妙之处在于，它既保留了绝对位置编码中的绝对位置信息，又保留了在内积运算下，对位置信息的相对性。

RoPE主要借助了复数的思想。为了引入复数，首先假设了在加入位置信息之前，原有的编码向量是二维行向量q_m和k_n，其中m和n是绝对位置，现在需要构造一个变换，将m和n引入到q_m和k_n中，即寻找变换：

$$\tilde{q_m} = f(q,m), \tilde{k_n} = f(k,n)$$

考虑到Attention的核心计算是内积：

$$Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

所以，寻求的这个$f(*)$变换，应该具有特性：$\langle f(q,m), f(k,n) \rangle = g(q,k,m-n)$

这里直接说结论，寻求的变换就是$q_m e^{im\theta}$，也就是给q_m乘以$e^{im\theta}$，相应地，k_n乘以$e^{in\theta}$。

具体的求解过程，请参考苏剑林老师的博客。

做了这样一个变换之后，根据复数的特性，有：

$$\langle q_m, k_n \rangle = Re[q_m k_n^*]$$

也就是，如果把二维向量看做复数，那么它们的内积，等于一个复数乘以另一个复数的共轭，得到的结果再取实部。

带入上面的变换，也就有：

$$\langle q_m e^{im\theta}, k_n e^{in\theta} \rangle = Re[(q_m e^{im\theta})(k_n e^{in\theta})^*] = Re[q_m k_n^* e^{i(m-n)\theta}]$$

这样一来，内积的结果就只依赖于$(m-n)$，也就是相对位置了。换言之，经过这样一番操作，通过给Embedding添加绝对位置信息，可以使得两个token的编码，经过内积变换（self-attn）之后，得到结果，是受它们位置的差值，即相对位置影响的。

于是对于任意的位置为m的二维向量$[x,y]$，把它看做复数，乘以$e^{im\theta}$，而根据欧拉公式，有：

$$e^{im\theta} = \cos m\theta + i \sin m\theta$$

于是上述的相乘变换也就变成了：

$$(x+iy)e^{im\theta} = (x\cos m\theta - y\sin m\theta) + i(x\sin m\theta + y\cos m\theta)$$

把上述式子写成矩阵形式：

$$f((q_0, q_1), m) = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \end{bmatrix}$$

而这个变换的几何意义，就是在二维坐标系下，对向量(q_0, q_1)进行了旋转，因而这种位置编码方法，被称为旋转位置编码。

根据刚才的结论，结合内积的线性叠加性，可以将结论推广到高维的情形。可以理解为，每两个维度一组，进行了上述的“旋转”操作，然后再拼接在一起：

$$\begin{bmatrix} \cos m\theta_0 & -\sin m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_0 & \cos m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_1 & -\sin m\theta_1 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_1 & \cos m\theta_1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2-1} & -\sin m\theta_{d/2-1} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2-1} & \cos m\theta_{d/2-1} \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \vdots \\ q_{d-2} \\ q_{d-1} \end{bmatrix}$$

由于矩阵的稀疏性，会造成计算上的浪费，所以在计算时采用逐位相乘再相加的方式进行：

$$\begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \vdots \\ q_{d-2} \\ q_{d-1} \end{bmatrix} \otimes \begin{bmatrix} \cos m\theta_0 \\ \cos m\theta_0 \\ \cos m\theta_1 \\ \cos m\theta_1 \\ \vdots \\ \cos m\theta_{d/2-1} \\ \cos m\theta_{d/2-1} \end{bmatrix} + \begin{bmatrix} -q_1 \\ q_0 \\ -q_3 \\ q_2 \\ \vdots \\ -q_{d-1} \\ q_{d-2} \end{bmatrix} \otimes \begin{bmatrix} \sin m\theta_0 \\ \sin m\theta_0 \\ \sin m\theta_1 \\ \sin m\theta_1 \\ \vdots \\ \sin m\theta_{d/2-1} \\ \sin m\theta_{d/2-1} \end{bmatrix}$$

其中$\otimes$为矩阵逐位相乘操作。代码中具体的计算过程，会有所出入，具体见下文。

3. 代码解读

3.1 tokenizer

tokenizer这部分没有太多可以讲的，主要就是用到了sentencepiece工具。

```python
from sentencepiece import SentencePieceProcessor
from logging import getLogger
from typing import List
import os


logger = getLogger()


class Tokenizer:
    def __init__(self, model_path: str):
        # reload tokenizer
        assert os.path.isfile(model_path), model_path
        self.sp_model = SentencePieceProcessor(model_file=model_path)
        logger.info(f"Reloaded SentencePiece model from {model_path}")

        # BOS / EOS token IDs
        self.n_words: int = self.sp_model.vocab_size()
        self.bos_id: int = self.sp_model.bos_id()
        self.eos_id: int = self.sp_model.eos_id()
        self.pad_id: int = self.sp_model.pad_id()
        logger.info(
            f"#words: {self.n_words} - BOS ID: {self.bos_id} - EOS ID: {self.eos_id}"
        )
        assert self.sp_model.vocab_size() == self.sp_model.get_piece_size()

    def encode(self, s: str, bos: bool, eos: bool) -> List[int]:
        assert type(s) is str
        t = self.sp_model.encode(s)
        if bos:
            t = [self.bos_id] + t
        if eos:
            t = t + [self.eos_id]
        return t

    def decode(self, t: List[int]) -> str:
        return self.sp_model.decode(t)
```

3.2 model

3.2.1 模型细节详解

model这部分的主要目的就是构建transformer，由于LLaMA对transformer在细节上做了一点改动，所以这里在介绍transformer部分之前，先结合前文模型细节介绍几个辅助函数：

（1）RMSNorm：

这部分的基本原理在上文中已经介绍过了，这里对代码部分进行简单的解释：

- x是输入
- weight是末尾乘的可训练参数
- x.pow(2)是平方
- mean(-1)实在最后一个维度（即hidden特征维度）上取平均
- eps防止取倒数之后分母为0
- torch.rsqrt是开平方并取倒数

结合上文的公式来看，是不难理解的。

```python
class RMSNorm(torch.nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def _norm(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)

    def forward(self, x):
        output = self._norm(x.float()).type_as(x)
        return output * self.weight
```

（2）RoPE旋转位置编码：

为了实现旋转位置编码，定义了三个辅助函数：

```python
def precompute_freqs_cis(dim: int, end: int, theta: float = 10000.0):
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() / dim))
    t = torch.arange(end, device=freqs.device)  # type: ignore
    freqs = torch.outer(t, freqs).float()  # type: ignore
    freqs_cis = torch.polar(torch.ones_like(freqs), freqs)  # complex64
    return freqs_cis


def reshape_for_broadcast(freqs_cis: torch.Tensor, x: torch.Tensor):
    ndim = x.ndim
    assert 0 <= 1 < ndim
    assert freqs_cis.shape == (x.shape[1], x.shape[-1])
    shape = [d if i == 1 or i == ndim - 1 else 1 for i, d in enumerate(x.shape)]
    return freqs_cis.view(*shape)


def apply_rotary_emb(
```

```
    xq: torch.Tensor,
    xk: torch.Tensor,
    freqs_cis: torch.Tensor,
) -> Tuple[torch.Tensor, torch.Tensor]:
    xq_ = torch.view_as_complex(xq.float().reshape(*xq.shape[:-1], -1, 2))
    xk_ = torch.view_as_complex(xk.float().reshape(*xk.shape[:-1], -1, 2))
    freqs_cis = reshape_for_broadcast(freqs_cis, xq_)
    xq_out = torch.view_as_real(xq_ * freqs_cis).flatten(3)
    xk_out = torch.view_as_real(xk_ * freqs_cis).flatten(3)
    return xq_out.type_as(xq), xk_out.type_as(xk)
```

这一部分是整个项目中，最不容易理解的部分，因为它跟一般的位置编码不同，即便是对transformer结构非常了解的同学，如果没有认真读过RoPE，对这一部分代码还是很难读明白。

看懂这一部分代码，最关键的是弄清楚其中的变量**freqs_cis**所指是什么东西。

为了搞懂这部分，我们需要先了解几个torch中不太常用的方法：

（1）torch.view_as_complex

把一个tensor转为复数形式，要求这个tensor的最后一个维度形状为2。

```
torch.view_as_complex(torch.Tensor([[1, 2], [3, 4], [5, 6]]))
# tensor([1.+2.j, 3.+4.j, 5.+6.j])
```

（2）torch.view_as_real

把复数tensor变回实数，可以看做是是刚才操作的逆变换。

```
torch.view_as_real(torch.view_as_complex(torch.Tensor([[1, 2], [3, 4], [5, 6]])))
# tensor([[1., 2.],
#         [3., 4.],
#         [5., 6.]])
```

（3）torch.outer

一个向量的转置乘以另一个向量：torch.outer(a, b) = a^T * b

```
a = torch.arange(1, 5)
b = torch.arange(1, 4)
torch.outer(a, b)
# tensor([[ 1,  2,  3],
#         [ 2,  4,  6],
#         [ 3,  6,  9],
#         [ 4,  8, 12]])
```

（4）torch.polar

torch.polar(abs, angle)利用一个绝对数值，和一个角度值，在极坐标下构造一个复数张量$abs * \cos(angle) + abs * \sin(angle)j$。

```
torch.polar(torch.tensor([1], dtype=torch.float64), torch.tensor([np.pi / 2], dtype=torch.f
# tensor([6.1232e-17+1.j], dtype=torch.complex128)
```

接下来进入RoPE的计算，首先为了更加具象的表达，我们在此对各个维度的尺寸进行假设，假设batch_size为2，seq_len固定为512，attention_head的数量为12，每个attention_head的维度为64，那么，对于输入到multi-head attn中的输入x_q的尺寸就是`(2, 512, 12, 64)`。

回到我们刚才提出的问题，**freqs_cis**所指是什么东西，其实它就是需要计算出来的$m\theta$也就是跟绝对位置相关的旋转的角度，在极坐标下对应的复数tensor。

而函数`precompute_freqs_cis`就是提前将这些旋转角度对应的tensor给创建出来，并可以重复利用。因为确定了序列的最大长度，所以这个tensor是固定死的。根据后续的数据流我们可以发现，在调用该函数时，传入的两个参数分别是attention_head的维度，以及最大长度的两倍，具象地，也就是`64`和`1024`。

我们逐行来理解这个方法：

```
freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() / dim))
```

首先torch.arange创建了一个tensor，$[0, 2, 4, ..., 60, 62]$，然后统一除以64，把它变成分数，然后整体作为基础角度的指数，它的shape是`(32)`

```
t = torch.arange(end, device=freqs.device)
```

t比较容易理解，也就是绝对位置信息，它的shape是`(1024)`。

```
freqs = torch.outer(t, freqs).float()
```

于是根据torch.outer运算，我们得到了一个shape为`(1024, 32)`的tensor。其意义也就是将每一个绝对位置，分配到对应的角度，相乘。直观理解一下，就是每一个绝对位置上，都有32个角度。为什么是这样的呢，回顾计算的公式，对于旋转矩阵，每两个元素为一组，它们乘以的角度是同一个θ，所以这个`(1024, 32)`，在后续的过程中，就可以reshape成`(512, 64)`，并且在64的那个维度上，每两个是相同的。

```
freqs_cis = torch.polar(torch.ones_like(freqs), freqs)
```

这一步就是在生成我们需要的位置信息，直观理解一下，像是在复平面内，以原点为中心，转了1024组，每一组64个的单位向量，它的shape是`(1024, 64)`。

`reshape_for_broadcast`方法，是把**freqs_cis**变成和输入的tensor相同的形状，结合下边的另一个方法一起介绍。

然后来看 `apply_rotary_emb` 方法，这个方法其实就是把位置信息添加到原有的编码结果上，在multi-head attention阶段调用。我们还是逐行来看：

```
xq_ = torch.view_as_complex(xq.float().reshape(*xq.shape[:-1], -1, 2))
```

上文中，我们假设了输入x_q的尺寸就是 `(2, 512, 12, 64)`，那么这一句操作的reshape，就是把它变成 `(2, 512, 12, -1, 2)`，也就是 `(2, 512, 12, 32, 2)`。x_k同理，略。紧接着把它变成复数形式，也就是变成了 `(2, 512, 12, 32)` 的形状。

然后进入到 `reshape_for_broadcast` 方法：

```
shape = [d if i == 1 or i == ndim - 1 else 1 for i, d in enumerate(x.shape)]
return freqs_cis.view(*shape)
```

这个方法的作用是为了把**freqs_cis**变成和输入的tensor相同的形状。需要注意的是，这里的freqs_cis**并不是** `precompute_freqs_cis` 生成的形状为 `(1024, 64)` 的那个tensor，而是根据输入的绝对位置，在 `(1024, 64)` 的tensor中，截取了长度为当前 `seq_len` 的一部分，代码在Transformer类的forward方法中：

```
freqs_cis = self.freqs_cis[start_pos : start_pos + seqlen]
```

也就是说，假如当前输入的序列长度是512，那么截取出来的这个新的 `freqs_cis`，形状就是 `(512, 64)`，reshape之后，形状就变成了 `(1, 512, 1, 32)`，也就是在每一个位置上，都对应有32个角度，根据刚刚torch.polar的介绍，当我们固定绝对值（也就是向量的模长）时，角度就可以在笛卡尔坐标系下唯一确定一个复数，这样一来也就是32个复数，即64个特征维度，所以就可以对应的将它融合到每个attention head的64个特征中去了。

reshape之后，就是将位置信息融入query和key中：

```
xq_out = torch.view_as_real(xq_ * freqs_cis).flatten(3)
```

这一步将二者相乘得到的复数tensor，重新转换为实数形式，得到的shape为 `(2, 512, 12, 32, 2)`，然后再flatten成 `(2, 512, 12, 64)`，这样一来，就变回了和最开始x_q相同的形状，也就完成了将位置信息融入到x_q的这一操作。x_k同理。

以上就是添加位置编码的整个过程，建议这一部分仔细阅读，反复理解。

至于SwiGLU激活函数，可以通过调用torch内置方法 `F.silu()` 实现，会在下文的FFN部分介绍。

3.2.2 transformer构建

接下来是transformer模型的构建。通常，我们在构建transformer时，是按Block构建的，每个transformer Block包含SA和FFN两部分，然后再通过堆叠block的形式，构建起整个transformer网络，LLaMA也是这样做的，读过BERT或者任何transformer结构的模型源码的同学一定对这个结构非常熟悉了。

首先看SA部分：

```python
class Attention(nn.Module):
    def __init__(self, args: ModelArgs):
        super().__init__()

        self.n_local_heads = args.n_heads // fs_init.get_model_parallel_world_size()
        self.head_dim = args.dim // args.n_heads

        self.wq = ColumnParallelLinear(
            args.dim,
            args.n_heads * self.head_dim,
            bias=False,
            gather_output=False,
            init_method=lambda x: x,
        )
        self.wk = ColumnParallelLinear(
            args.dim,
            args.n_heads * self.head_dim,
            bias=False,
            gather_output=False,
            init_method=lambda x: x,
        )
        self.wv = ColumnParallelLinear(
            args.dim,
            args.n_heads * self.head_dim,
            bias=False,
            gather_output=False,
            init_method=lambda x: x,
        )
        self.wo = RowParallelLinear(
            args.n_heads * self.head_dim,
            args.dim,
            bias=False,
            input_is_parallel=True,
            init_method=lambda x: x,
        )

        self.cache_k = torch.zeros(
            (args.max_batch_size, args.max_seq_len, self.n_local_heads, self.head_dim)
        ).cuda()
        self.cache_v = torch.zeros(
            (args.max_batch_size, args.max_seq_len, self.n_local_heads, self.head_dim)
        ).cuda()

    def forward(self, x: torch.Tensor, start_pos: int, freqs_cis: torch.Tensor, mask: Optio
        bsz, seqlen, _ = x.shape
        xq, xk, xv = self.wq(x), self.wk(x), self.wv(x)

        xq = xq.view(bsz, seqlen, self.n_local_heads, self.head_dim)

```

```python
        xk = xk.view(bsz, seqlen, self.n_local_heads, self.head_dim)
        xv = xv.view(bsz, seqlen, self.n_local_heads, self.head_dim)

        xq, xk = apply_rotary_emb(xq, xk, freqs_cis=freqs_cis)

        self.cache_k = self.cache_k.to(xq)
        self.cache_v = self.cache_v.to(xq)

        self.cache_k[:bsz, start_pos : start_pos + seqlen] = xk
        self.cache_v[:bsz, start_pos : start_pos + seqlen] = xv

        keys = self.cache_k[:bsz, : start_pos + seqlen]
        values = self.cache_v[:bsz, : start_pos + seqlen]

        xq = xq.transpose(1, 2)
        keys = keys.transpose(1, 2)
        values = values.transpose(1, 2)
        scores = torch.matmul(xq, keys.transpose(2, 3)) / math.sqrt(self.head_dim)
        if mask is not None:
            scores = scores + mask  # (bs, n_local_heads, slen, cache_len + slen)
        scores = F.softmax(scores.float(), dim=-1).type_as(xq)
        output = torch.matmul(scores, values)  # (bs, n_local_heads, slen, head_dim)
        output = output.transpose(
            1, 2
        ).contiguous().view(bsz, seqlen, -1)

        return self.wo(output)
```

这一部分看上去会比较复杂，涉及到了很多的计算，但其实它就是最普通的attention，只要牢记attention的核心计算公式，也不难理解。

其中，为了执行多卡并行，这里的Linear层用的都是fairscale中的类，在阅读代码时直接理解为Linear即可。

attention计算的总体过程是：

- 1. 输入x，分别经过三个Linear得到x_q , x_k , x_v ；
- 2. 在x_q 和x_k 中加入旋转位置编码；
- 3. 缓存x_q 和x_k ；
- 4. 计算$softmax(\frac{QK^T}{\sqrt{d_k}})V$。

其中有一个细节就是缓存机制，这里简单介绍一下，很多初学者，甚至NLP老手都容易忽视这个问题。这个机制在模型的训练过程中其实是不发挥作用的，它设计的目的是在generate时减少token的重复计算。

简单解释一下，就是在计算第n个token特征的时候，需要用到第$1, ..., n-1$个token，即每次生成时，需要知道前面所有的过往信息，如果每次都从头算的话，那就会造成极大的浪费，所以就没算一个位置的信息，就把

它缓存下来。

然后是FFN部分，需要注意的点就是采用的激活函数，以及激活函数的位置：

```python
class FeedForward(nn.Module):
    def __init__(
        self,
        dim: int,
        hidden_dim: int,
        multiple_of: int,
    ):
        super().__init__()
        hidden_dim = int(2 * hidden_dim / 3)
        hidden_dim = multiple_of * ((hidden_dim + multiple_of - 1) // multiple_of)

        self.w1 = ColumnParallelLinear(
            dim, hidden_dim, bias=False, gather_output=False, init_method=lambda x: x
        )
        self.w2 = RowParallelLinear(
            hidden_dim, dim, bias=False, input_is_parallel=True, init_method=lambda x: x
        )
        self.w3 = ColumnParallelLinear(
            dim, hidden_dim, bias=False, gather_output=False, init_method=lambda x: x
        )

    def forward(self, x):
        return self.w2(F.silu(self.w1(x)) * self.w3(x))
```

这里与常见模型中的FFN做一下简单的对比，BART中的FFN，用的是`fc->act->fc`，用了两层全连接；GPT中的FFN，用的是`conv1D->act->conv1D`，也是只用了两层。

而LLaMA中的FFN采用了三个全连接层以实现FFNSwiGLU，即

$$FFN_{swiGLU}(x, W, V, W_2) = (Swish_1(xW) \otimes xV)W_2$$

然后将SA和FFN这两部分拼在一起就是一个transformer block

```python
class TransformerBlock(nn.Module):
    def __init__(self, layer_id: int, args: ModelArgs):
        super().__init__()
        self.n_heads = args.n_heads
        self.dim = args.dim
        self.head_dim = args.dim // args.n_heads
        self.attention = Attention(args)
        self.feed_forward = FeedForward(
            dim=args.dim, hidden_dim=4 * args.dim, multiple_of=args.multiple_of
```

```
        )
        self.layer_id = layer_id
        self.attention_norm = RMSNorm(args.dim, eps=args.norm_eps)
        self.ffn_norm = RMSNorm(args.dim, eps=args.norm_eps)

    def forward(self, x: torch.Tensor, start_pos: int, freqs_cis: torch.Tensor, mask: Optio
        h = x + self.attention.forward(self.attention_norm(x), start_pos, freqs_cis, mask)
        out = h + self.feed_forward.forward(self.ffn_norm(h))
        return out
```

最后利用torch的module list将transformer block进行堆叠，拼上最前头的embedding部分，就是一个完整的transformer（decoder）结构了。

```
class Transformer(nn.Module):
    def __init__(self, params: ModelArgs):
        super().__init__()
        self.params = params
        self.vocab_size = params.vocab_size
        self.n_layers = params.n_layers

        self.tok_embeddings = ParallelEmbedding(
            params.vocab_size, params.dim, init_method=lambda x: x
        )

        self.layers = torch.nn.ModuleList()
        for layer_id in range(params.n_layers):
            self.layers.append(TransformerBlock(layer_id, params))

        self.norm = RMSNorm(params.dim, eps=params.norm_eps)
        self.output = ColumnParallelLinear(
            params.dim, params.vocab_size, bias=False, init_method=lambda x: x
        )

        self.freqs_cis = precompute_freqs_cis(
            self.params.dim // self.params.n_heads, self.params.max_seq_len * 2
        )

    @torch.inference_mode()
    def forward(self, tokens: torch.Tensor, start_pos: int):
        _bsz, seqlen = tokens.shape
        h = self.tok_embeddings(tokens)
        self.freqs_cis = self.freqs_cis.to(h.device)
        freqs_cis = self.freqs_cis[start_pos : start_pos + seqlen]

        mask = None
        if seqlen > 1:
            mask = torch.full((1, 1, seqlen, seqlen), float("-inf"), device=tokens.device)
            mask = torch.triu(mask, diagonal=start_pos + 1).type_as(h)


```

```python


        for layer in self.layers:
            h = layer(h, start_pos, freqs_cis, mask)
        h = self.norm(h)
        output = self.output(h[:, -1, :])  # only compute last logits
        return output.float()
```

直接看forward部分，输入是token，先做token embedding，然后添加位置信息。对于decoder模型，为了防止标签泄漏，需要mask，所以做了一个上三角的mask矩阵。接下来就是逐层的计算transformer。

3.3 generate

```python
class LLaMA:
    def __init__(self, model: Transformer, tokenizer: Tokenizer):
        self.model = model
        self.tokenizer = tokenizer

    def generate(
        self,
        prompts: List[str],
        max_gen_len: int,
        temperature: float = 0.8,
        top_p: float = 0.95,
    ) -> List[str]:
        bsz = len(prompts)
        params = self.model.params
        assert bsz <= params.max_batch_size, (bsz, params.max_batch_size)

        prompt_tokens = [self.tokenizer.encode(x, bos=True, eos=False) for x in prompts]

        min_prompt_size = min([len(t) for t in prompt_tokens])
        max_prompt_size = max([len(t) for t in prompt_tokens])

        total_len = min(params.max_seq_len, max_gen_len + max_prompt_size)

        tokens = torch.full((bsz, total_len), self.tokenizer.pad_id).cuda().long()
        for k, t in enumerate(prompt_tokens):
            tokens[k, : len(t)] = torch.tensor(t).long()
        input_text_mask = tokens != self.tokenizer.pad_id
        start_pos = min_prompt_size
        prev_pos = 0
        for cur_pos in range(start_pos, total_len):
            logits = self.model.forward(tokens[:, prev_pos:cur_pos], prev_pos)
            if temperature > 0:
                probs = torch.softmax(logits / temperature, dim=-1)
                next_token = sample_top_p(probs, top_p)
            else:
                next_token = torch.argmax(logits, dim=-1)
            next_token = next_token.reshape(-1)
```

```python
            # only replace token if prompt has already been generated
            next_token = torch.where(
                input_text_mask[:, cur_pos], tokens[:, cur_pos], next_token
            )
            tokens[:, cur_pos] = next_token
            prev_pos = cur_pos

        decoded = []
        for i, t in enumerate(tokens.tolist()):
            # cut to max gen len
            t = t[: len(prompt_tokens[i]) + max_gen_len]
            # cut to eos tok if any
            try:
                t = t[: t.index(self.tokenizer.eos_id)]
            except ValueError:
                pass
            decoded.append(self.tokenizer.decode(t))
        return decoded

def sample_top_p(probs, p):
    probs_sort, probs_idx = torch.sort(probs, dim=-1, descending=True)
    probs_sum = torch.cumsum(probs_sort, dim=-1)
    mask = probs_sum - probs_sort > p
    probs_sort[mask] = 0.0
    probs_sort.div_(probs_sort.sum(dim=-1, keepdim=True))
    next_token = torch.multinomial(probs_sort, num_samples=1)
    next_token = torch.gather(probs_idx, -1, next_token)
    return next_token
```

生成的过程如下：

- 1. 对prompts进行tokenize，得到token ids；
- 2. 计算当前batch的最大长度total_len，用来创建输入的token tensor，最大长度不能超过前文所述缓存的大小；
- 3. 从当前batch中，最短的一个prompt的位置，作为生成的开始位置，开始生成；
- 4. 输入的token tensor传入transformer模型，计算logits，得到形状为`(batch_size, hidden_size)`的logits（transformer最后一层的输出）；
- 5. softmax+top_p采样，得到当前预测的token，并更新当前位置，准备预测下一个token；
- 6. 解码得到生成的文本。

4. 推理

简单看一下官方example中给出的推理样例prompt：

```
['The capital of Germany is the city of',
 'Here is my sonnet in the style of Shakespeare about an artificial intelligence:']
```

生成结果为：

```
['The capital of Germany is the city of Berlin. The city is also the capital of the Federal
 'Here is my sonnet in the style of Shakespeare about an artificial intelligence:\nLet us t
```

总结一下，本文对LLaMA大模型的结构代码进行了详细的介绍，其开源出来的结构代码量并不多，但是其中很多细节值得反复推敲理解。

在后续的工作中，可能会对大模型进行进一步的实验，对此欢迎对此感兴趣的朋友们在下方留言交流。如果本文中出现了不够准确的地方，也欢迎大家在评论区指出。

文章知识点与官方知识档案匹配，可进一步学习相关知识

Python入门技能树 > 人工智能 > 深度学习　405317 人正在系统学习中